

Objects as Software Services

Gilad Bracha

Bit Rot

- Bits don't rot
- It would be better if they did
- Dynamically typed languages can help us rot them

Software requires Maintenance

- We expect software updates with
 - bug fixes
 - new features
- Dynamic update is standard practice for, e.g., OS vendors

First, Pester



Then, ask for ID

Authenticate

 Software Update requires that you type your password.

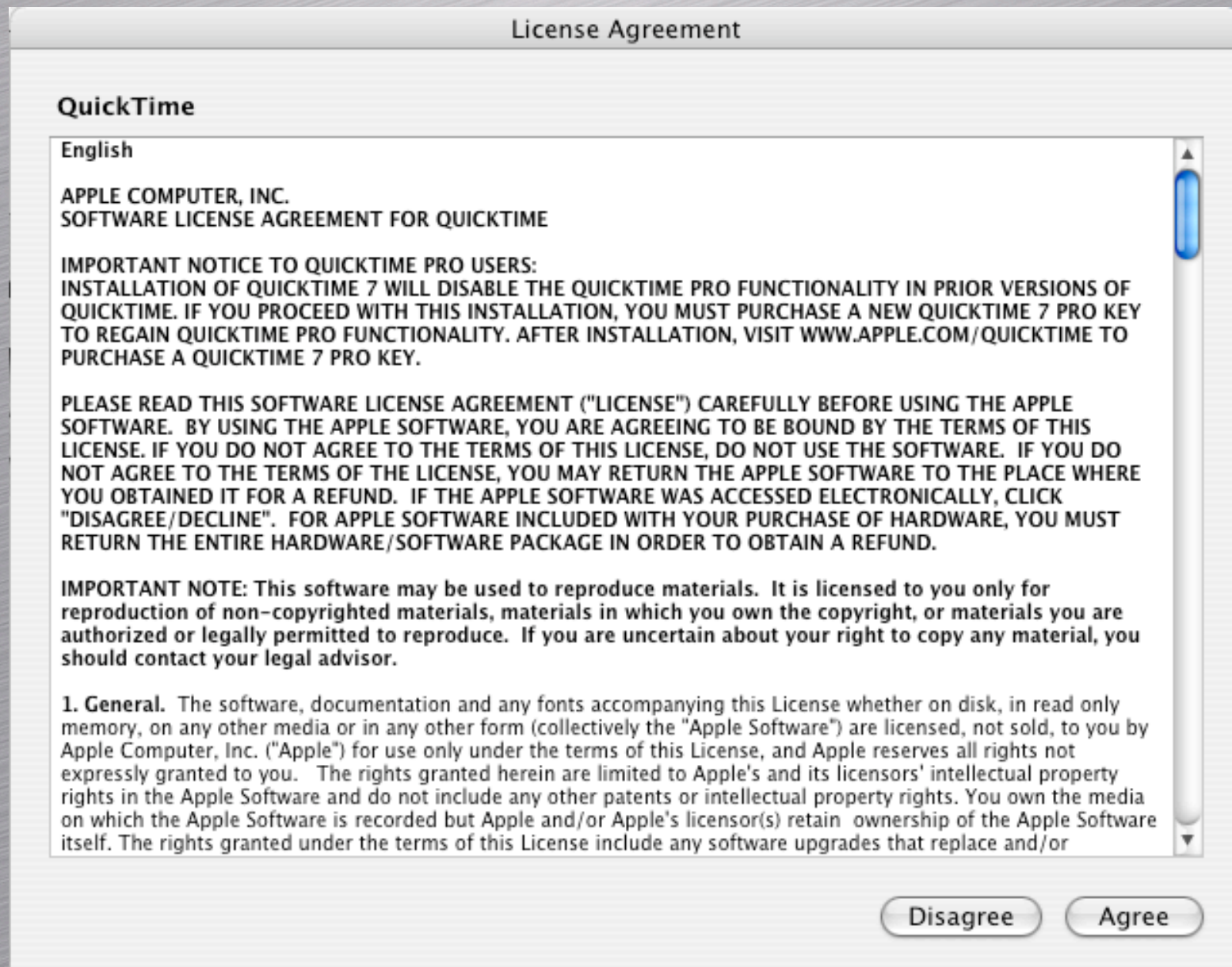
Name:

Password:

► Details



Then, legalese



Then, the coup de grace



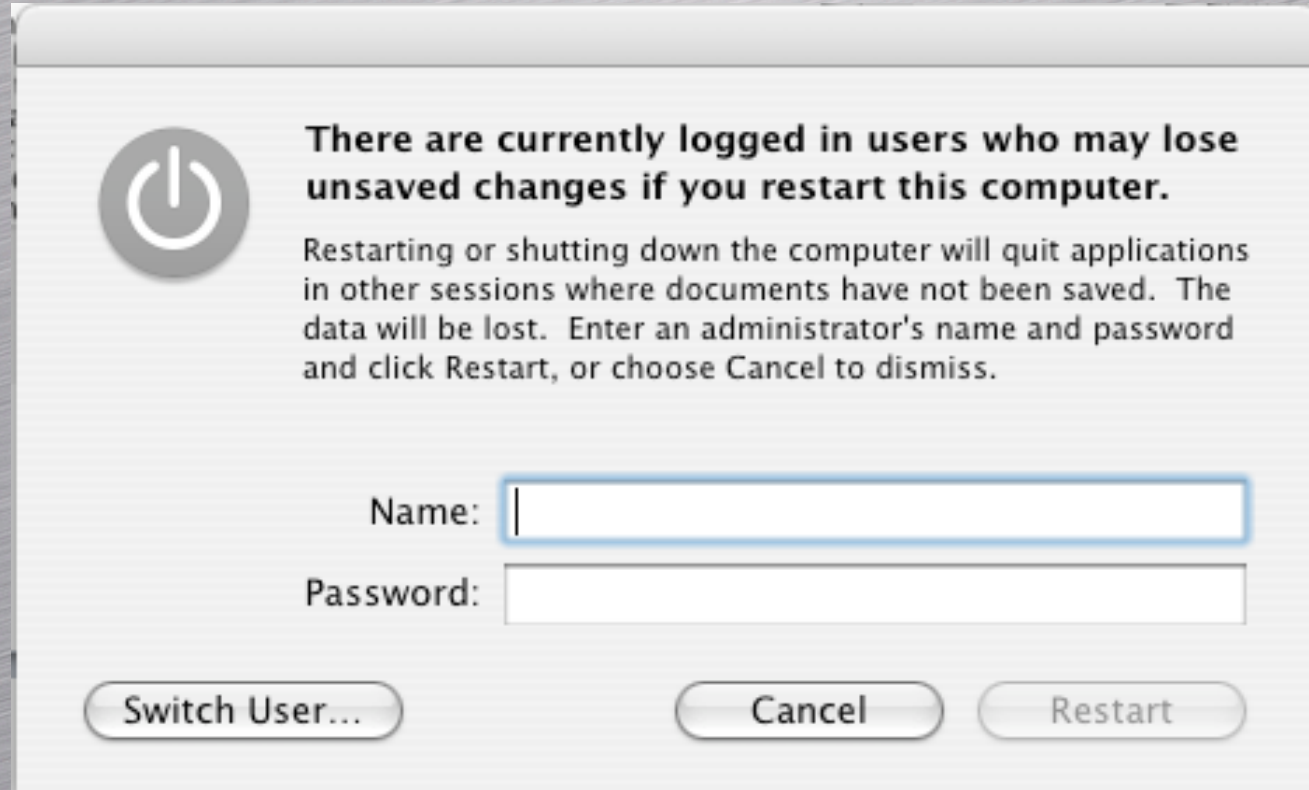
The new software requires that you restart your computer now.

Click Restart to quit all applications and restart.

Shut Down

Restart

But wait, there's more



A screenshot of a Mac OS X shutdown dialog box. It features a power button icon on the left. The main text warns that restarting will lose unsaved changes and prompts for administrator credentials. Below the text are input fields for 'Name' and 'Password'. At the bottom are three buttons: 'Switch User...', 'Cancel', and 'Restart'.

 **There are currently logged in users who may lose unsaved changes if you restart this computer.**

Restarting or shutting down the computer will quit applications in other sessions where documents have not been saved. The data will be lost. Enter an administrator's name and password and click Restart, or choose Cancel to dismiss.

Name:

Password:

Expect better

- Make maintenance as transparent as possible
 - No questions, hassles
 - Nothing should ever boot or reboot
- Always up to date
 - Like a web app/service

Web Apps have downsides

- System software has to be local
- UI issues
- Depend on network being:
 - Reliable
 - Fast
 - Cheap
- Still make you “reboot” - it’s called:
session expired

Software Services

- Combine advantages of web services and traditional client applications
 - Always Available (even w/o network)
 - Always Up to date
- Run locally, think globally

Dynamically Typed Languages to the Rescue

- Lots of experience with updating code on the fly
- Much easier to do in the absence of mandatory static type system

Self Modifying Code

- Makes people nervous
 - Lots of issues:
 - What happens if the modified code is still active (on the stack)
 - What happens to instances of modified classes
 - schema changes, representation invariants
 - Security

Self Modifying Code

- Needs structure
 - Mirror based Reflection
- Much easier if program is quiescent
 - This does not mean waiting until the program restarts.

When is the Program Quiescent?

- Many applications perform data synchronization over the network
- Synchronizing with server
 - Provides reliable backup, audit trail
 - Allows access from multiple devices
 - Supports collaboration

Synchronization

- Natural point for program update
 - Applications are quiescent
 - Transition is user-visible
- Program as Data
 - Sync program as well as data

Orthogonal Synchronization

- All persistent data is sync'ed
 - Data is persistent if it is reachable from a persistent root, and not marked transient
- Transient data is lazily recomputed after every sync
 - This can be enforced with aid of context free syntax, e.g., *transient f [initExpr]*

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply
 - *Data outlives Program:*

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply
- ~~*Data outlives Program:*~~ Program and data live as long as the service

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply
- ~~*Data outlives Program:*~~ Program and data live as long as the service
- *Transient data pollutes database:*

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply
- ~~*Data outlives Program:*~~ Program and data live as long as the service
- ~~*Transient data pollutes database:*~~ transient data is zapped at every sync

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply
- ~~*Data outlives Program:*~~ Program and data live as long as the service
- ~~*Transient data pollutes database:*~~ transient data is zapped at every sync
- *No cross-program interchange format:*

Orthogonal Synchronization

- Criticisms of orthogonal persistence do not apply
- ~~**Data outlives Program:**~~ Program and data live as long as the service
- ~~**Transient data pollutes database:**~~ transient data is zapped at every sync
- ~~**No cross-program interchange format:**~~ XML

Orthogonal Synchronization

- Efficient synchronization requires knowing what has changed
- Persistent objects should log changes
- A good language will ensure that all access is mediated by getters/setters
- When an object becomes persistent, change its setters so that they log changes

Key Points

Dynamic Typing



Hotswapping



Orthogonal Synchronization



Software Services

Security

- Hotswapping based upon network input is scary
- Must verify identity of server and clients
- Need strong security model for code
 - Can we achieve this w/o typed assembly language/wire format?

Security

- Wire format must be
 - dynamically type safe
 - pointer safe
- Objects must be strongly encapsulated
 - Objects will serve as capabilities

Security

- Mirrors act as capabilities for reflection
 - Provide single, centralized access to all reflective operations
 - Specific subsets available through particular mirrors
- Not something you can get from traditional reflective API or from popular scripting languages

Security

- Strong sandbox - no global/static state
(aka *No Ambient Authority*)

Modules

- For Development & Deployment
- No static state
 - Global internet-style namespace for immutables only
- No versions

Modules

- For Development & Deployment
- No static state
 - Global internet-style namespace for immutables only
- No versions

Modules

- Self contained parametric namespace
 - No imports
 - imports are for localizing couplings, not for decoupling
- All external dependencies are pluggable parameters
- Only parameter declarations see surrounding namespace
- Explicit export of module elements

Modules

- Are instantiated into stateful objects
- Top level module instantiation happens in namespaces with access to globals
- Parameters are objects/capabilities that determine per-module sandbox

Speculative Syntax

```
main(platform, args) {  
    letrec  
        app = new com.foo.bar.demo(sandbox, args);  
        sandbox = platform.restrictedSandbox();  
    in app.run();  
}
```


Modules

- For Development & Deployment
- No static state
 - Global internet-style namespace for immutables only
- No versions

No Static

- Classes, Modules, Namespaces are values (and so are numbers, ...)
- Good for
 - Distribution
 - Security
 - Startup
 - Memory management

Modules

- For Development & Deployment
- No static state
 - Global internet-style namespace for immutables only
- No versions

No Versions

- Users subscribe to a software service
- Bug fixes and updates included in subscription
- Only one current version at any time
- No releases!

No Releases?!

- This is a radical change in the development model
- Relatively easy for applications
- Hard for libraries and components
 - How do we deal with incompatibilities

No Releases!

- Developers subscribe to pre-release libraries
- Change cycle is very rapid - days/weeks rather than months/years
- Expect libraries to morph on you daily, and be prepared to adapt
- Development model is more like open source: ***Bits Rot, deal with it***

No Releases/Versions

- Can this work?
- As a producer of incompatible code you can find out if anyone cares
 - Do *senders-of* globally on the entire planet
 - Refactor callers
 - If anything breaks worldwide - you'll know.

No Releases/Versions

- As a consumer of an incompatible API you can respond rapidly
- Manage transition with conditional code - and get rid of the mess the next day.
- If anything breaks worldwide - you'll know.
- Bugs that aren't caught in development can still be fixed almost immediately.

Connections

- Mirrors
 - Self
 - Strongtalk, JDI, APT ... See OOPSLA 04
- No static
 - Scala
 - Fortress
 - E

Connections

- Security

- E

- Java

- Modules

- Jigsaw, 1991

- Units

- ML

- Fortress

Connections

- Representation independence
 - Self
- Networked Clients
 - Rich, Thin, Fat, Smart, Managed ...
 - AJAX
 - Flash
 - Avalon, XAML
 - dotmac
 - Many others ...

Connections

- Synchronization and networked stores
 - SyncML
 - WebDAV

Summary

Object based Encapsulation

Dynamic Typing

Security

Hotswapping

Software Services

Version free Software

Rotting Bits for a better World

- The indestructability of bits is a hidden curse
- A model which expects incompatibility as a matter of course is better than denying change
- Dynamically typed, secure, modular languages can enable such a model

Backups

Semantics were quite different

Better methods of fast and fearless prototyping and development are needed. The Java language environment is one of those better ways, because it's *interpreted* and *dynamic*.

- *The Java Language Environment :A White Paper*
James Gosling and Henry McGilton

Semantics were quite different

Programmers using “traditional” software development tools have become resigned to the artificial edit-compile-link-load-throw-the-application-off-the-cliff-let-it-crash-and-start-all-over-again style of current development practice.

- *The Java Language Environment :A White Paper*
James Gosling and Henry McGilton

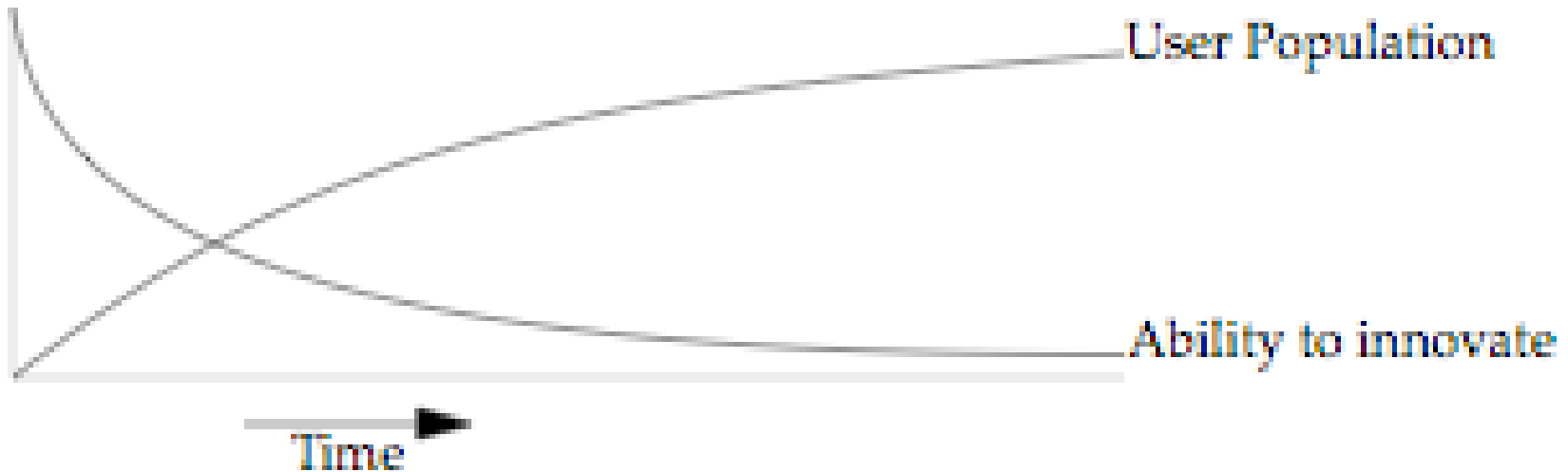
One of the goals of JAVA is to enable the construction of software that can run stand-alone in small machines. The size of the basic interpreter and class support is about 30 Kbytes, adding the basic standard libraries and thread support (essentially a self-contained microkernel) brings it up to about 120K.

*- The Java Language Environment :A White Paper
James Gosling and Henry McGilton*

Innovation on the Internet follows a pattern:
initially: someone develops a technology.
They're free to try all kinds of things since no
one else is using the technology and there are
no compatibility issues. Slowly, people start
using it, and as they do, compatibility and
interoperability concerns slow the pace of
innovation.

- The Java Language Environment :A White Paper
James Gosling and Henry McGilton

Innovation vs. Popularity



- *The Java Language Environment :A White Paper*
James Gosling and Henry McGilton

Within a community that uses HotJava, individuals can experiment with new facilities while at the same time preserving compatibility and interoperability. Data can be published in new formats and distributed using new protocols and the implementations of these will be automatically and safely installed. **There is no upgrade problem.**

- The Java Language Environment :A White Paper
James Gosling and Henry McGilton